

Introduction of Object Oriented Programming

What is Programming Language?

- Programming Language is a platform where we can arrange particular command list to perform such task in the form of program.
- Example of programming languages like,
 1. C,
 2. C++,
 3. C#.net,
 4. VB.net etc...

Types of Programming Language.

- In human concept, there are two types of languages like:
 - Symbolic Language
 - Words Language
 - like, English, Gujarati, Hindi etc.
- Same as that there are two types of programming languages available in Computer :
 - POP
 - OOP

What is POP?

POP stands for,
Procedure
Oriented
Programming

Example of Procedure Oriented Programming Language :

C
VB
FORTRAN
Pascal etc.

Procedure Oriented Programming :

■ What is Procedure?

- The process which gives multiple works at a time is known as procedure.
- **In computer meaning:**
 - Procedure means to execute all the code of program at a time.
- **Simple meaning:**
 - Procedure is a block which can execute all the code of a task at a time.

Characteristics of POP :

- The main focus is on algorithms.
- Complex programs are divided into function
- Global data is shared by functions.
- There is no protection on data.
- It follows top-down approach.

■ Limitations of POP

- Time consuming because it works till starting to ending of process.
- All code of program will get process at a time of execution when those codes are not necessary.

What is OOP?

- OOP stands for,
 - ❑ **O**bject
 - ❑ **O**riented
 - ❑ **P**rogramming
- Object oriented programming (**OOP**) is updated programming from Procedure Oriented Programming (**POP**).
- **Example of Object Oriented Programming.**
 - ❑ C++, JAVA, VB.NET, C#.NET etc.
- The object oriented programming approach came with major modifications in procedure oriented programming.
- First of all the data is protected in OOP.
- In OOP approach, the large and complex program is divided into **objects** in-place of function of POP.
- Each object is associated with its class and the data is accessible within the class.
- Any function from outside the class cannot access the data of Class.

OOP Characteristics :

- In OOP the importance is on **data** in place of function.
- Large programs are divided into **objects**.
- The data of objects is hidden and can't be accessed by outside functions.
- Objects can pass information through functions.
- It follows Bottom-Up approach.

Difference between POP and OOP

POP	OOP
POP means Procedure Oriented Programming	OOP means Object Oriented Programming
Larger programs are divided into functions	Larger programs are divided into objects
In POP Functions are basic entity	In OOP Objects are basic entity
Functions share global data	Data is hidden and can't be accessed by outside functions
Follows Top-Down approach	Follows Bottom-Up approach
Examples : C, Cobol, Fortran	Examples : C++, C#, Java etc

Features of OOP : (Basic Concept Of OOP) :

- Objects
- Classes
- Data Abstraction
- Encapsulation
- Inheritance
- Polymorphism

Benefits of OOP :

- It saves program development time and gives higher productivity.
- Repeated code can be removed using inheritance.
- Secure programs can be built using data hiding.
- Large program can be partitioned based on objects.
- Complexity can be reduced.
- The system developed under object oriented model can be easily upgraded.

What is C++ ?

- Basically C++ is not a new language but it is just an updation to C programming language.
- C++ is an object oriented programming language.
- C++ is developed by Bjarne Stroustrup at AT&T Bell Laboratories in New Jersey, USA in the early 1980s.
- Earlier the name of the language was “C with classes” but after sometimes it was renamed to C++.
- ++ is an increment operator of C language.
- Here, C++ is an increment to the C language.

Structure of C :

First we revise the structure of C programming before understand structure of C++.

Structure of C Programming

1. Documentation.
2. File Include Section.
3. Symbolic Constant Definition
4. Global Variable Declaration
5. Main Function.
6. User Defined Function.

Structure of C++ Programming

1. Documentation.
2. File Include Section.
3. Global Variable Declaration.
4. Class Declaration.
5. Function Definition Section.
6. Main Function.

Output Operator :

- Output Data Using cout
- In C++ we can use cout operator to display data on screen.
- The output operator (<<) is also known as insertion operator because it inserts the text or value of the variable passed with it.

Syntax :

`cout<<"string" or variable;`

Example :

```
cout<<"CREATIVE MULTIMEDIA";  
cout<<val1;
```

Input Operator :

- **Input Data Using cin**
- **In C++ we can use cin operator to enter some values to a variable.**
- **The input operator (>>) is also known as extraction operator because it extracts the text or value to the variable.**

Syntax : cin>>variable1>>variable2;

Example : cin>>a;

simple example of standard Input/output stream :

```
#include <iostream>
int main( )
{
    int a;
    cout<<"Enter A=";
    cin>>a;
}
```

Cout = for print msg

Cin>> for input value

<iostream>	It is used to define the cout, cin and objects, which correspond to standard output stream, standard input stream and standard, respectively.
------------	---

Tokens :

- Tokens are the building blocks of the C++ program.
- Every single CHARACTER of C++ program file can be considered as a C++ token.
- A token is a group of characters that logically belong together. The programmer can write a program by using tokens

Types of Token.



Keywords...

auto	declare a local variable
break	break out of a loop
case	a block of code in a switch statement
catch	handles exceptions from throw
char	declare a character variable
class	declare a class

const	declare immutable data or functions that do not change data
continue	bypass iterations of a loop
default	default handler in a case statement
delete	make memory available
do	looping construct
double	declare a double precision floating-point variable
else	alternate case for an if statement
enum	create enumeration types
float	declare a floating-point variable
for	looping construct
friend	grant non-member function access to private data
goto	jump to a different part of the program
if	execute code based off of the result of a test
int	declare a integer variable
long	declare a long integer variable
new	allocate dynamic memory for a new variable
private	declare private members of a class
protected	declare protected members of a class
public	declare public members of a class
return	return from a function
short	declare a short integer variable

signed	modify variable type declarations
sizeof	return the size of a variable or type
static	create permanent storage for a variable
struct	define a new structure
switch	execute code based off of different possible values for a variable
this	a pointer to the current object
throw	throws an exception
try	execute code that can throw an exception
union	a structure that assigns multiple variables to the same memory location
unsigned	declare an unsigned integer variable
virtual	create a function that can be overridden by a derived class
void	declare functions or data with no associated data type
while	looping construct

Identifiers ...

- An identifier is a name given to a variable, function, class, object, structure, union, enumeration.
- Identifier is a label to identify it in the program.
- We can give any name as an identifier but there are some rules to declare an identifier.
 - Keywords cannot be used as an identifier.
 - White space is not allowed.
 - No any special symbol except underscore (_) is allowed.
 - Digits are allowed but identifier should not start with a digit.
 - Uppercase and lowercase letters are different.

Constants :

- As the name suggests, constants are fixed values that should not be changed throughout the program execution.
- Constants can be integer, floating point, characters or strings.
- For Example,
 - Integer Constant : 123, 555, 2569 etc
 - Floating Point Constant : 1.23, 40.10, 1.75 etc
 - Character Constant: 'a', 'b', 'x', 'Y' etc
 - String Constant : "ABC", "Computer" etc

String :

- String are the words you write in the program such as in printing message or in string constants.

Special Symbol :

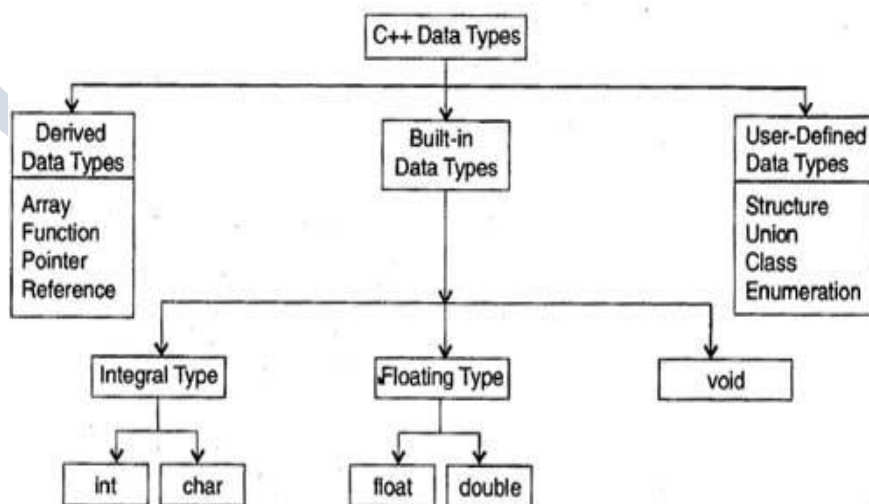
- Special Symbols are those characters which are used in the program to perform various tasks.

Operators :

- In the program, we can use operators in expressions such as : `c=a+b`
- Here, `+` and `=` both are operators

Data Types in C++ :

- We always use different types of data in our programming.
- We have various types of data. Like, `Sr_No`, `Name`, `Weight`, `age` etc.
- All data has its own data type.
- Data types means type of data to store in specified data type.
- **For example,**
 - We can store `Sr_No` in Integer Data Type
 - We can store `Weight` in Floating Data Type



➤ There are main three categories of data types.

➤ **Basic Data Types (Built-in)**

➤ **User Defined Data Types**

➤ **Derived Data Types**

Basic Data Types :

➤ Basic Data Types, also known as fundamental or built in data types of C++ are

➤ Integer

➤ Floating Point

➤ Void

➤ **Basic Data Types :**

➤ **Integer :**

➤ The variables of floating point data types can store the numeric values without any fractional part.

➤ Char, int, long are main integer

➤ **Floating point :**

➤ The variable of floating point data types can store the numeric values with fractional part.

➤ Float, double, long double are floating point data type.

➤ **Void :**

➤ The void data type is follows its name, it doesn't store any value.

Data Type	Size in Bytes	Range
char	1	-128 to 127
unsigned char	1	0 to 255 (ascii)
int	2	-32768 to 32767
unsigned int	2	0 to 65535
long	4	-2147483648 to 2147483674
float	4	3.4E-38 to 3.4E 38
double	8	1.7E-308 to 1.7E 308

User Define Data Types :

- Structure
- Unions
- Class
- Enumerations *are user define data types.*

Structure & Union :

Structure and union are user defined data type as they are defined by user.

Classes :

Class is another user defined data type of C++. It offers some more functionalities than a structure.

Enumerations

Using enumeration, we can enumerate a list of values. It assign 0 to first value, 1 to second value and so on.

➤ Example

```
enum fruits{apple, mango, grapes};
```

Here, apple will be assigned with 0
mango 1 and grapes 2

We can also assign specified number to particular item to...

```
Enum week{mon=1, tue, wed, thu, fri, sat, sun};
```

Example : Let's create a simple program of enum.

```
#include <stdio.h>
enum weekdays{Sunday=1, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday};
int main()
{
    enum weekdays w; // variable declaration of weekdays type
    w=Monday; // assigning value of Monday to w.
    printf("The value of w is %d",w);
    return 0;
}
```

In the above code, we create an enum type named as weekdays, and it contains the name of all the seven days. We have assigned 1 value to the Sunday, and all other names will be given a value as the previous value plus one.

Example 2:

```
#include <stdio.h>
enum months{jan=1, feb, march, april, may, june, july, august, september, october, november, december};
int main()
{
// printing the values of months
for(int i=jan;i<=december;i++)
{
printf("%d, ",i);
}
return 0;
}
```

Why do we use enum?

The enum is used when we want our variable to have only a set of values. For example, we create a direction variable. As we know that four directions exist (North, South, East, West), so this direction variable will have four possible values. But the variable can hold only one value at a time.

The enum is also used in a switch case statement in which we pass the enum variable in a switch parenthesis. It ensures that the value of the case block should be defined in an enum.

```
#include <stdio.h>
enum days{sunday=1, monday, tuesday, wednesday, thursday, friday, saturday};
int main()
{
enum days d;
d=monday;
switch(d)
{
case sunday:
printf("Today is sunday");
}
```



```
break;
case monday:
printf("Today is monday");
break;
case tuesday:
printf("Today is tuesday");
break;
case wednesday:
printf("Today is wednesday");
break;
case thursday:
printf("Today is thursday");
break;
case friday:
printf("Today is friday");
break;
case saturday:
printf("Today is saturday");
break;
}

return 0;
}
```

Derived Data Types :

Array, pointers, functions and reference variable are considered as derived data types. Derived Data Types are derived from the basic data types.

Symbolic Constants :

- In C++ we can create symbolic constant by many ways :
 - Using the const keyword
 - Using enum keyword
 - Using #define directive...
- Constant cannot redefine during program execution.

C const keyword :

The const keyword is used to define constant in C programming.

```
const float PI=3.14;
```

Now, the value of PI variable can't be changed.

Example :

```
#include<stdio.h>
int main(){
    const float PI=3.14;
    printf("The value of PI is: %f",PI);
    return 0;
}
```

C #define :

The #define preprocessor directive is used to define constant or micro substitution. It can use any basic data type.

Syntax:

#define token value

Let's see an example of #define to define a constant.

```
#include <stdio.h>
#define PI 3.14
main() {
    printf("%f",PI);
}
```

Sizeof Operator :

- To check the size of variable in bytes we can use sizeof operator.

For Example,

```
int a=150;
float b=25000;
cout<<"Size of A is "<<sizeof(a);
printf<<"Size of B :%d"<<sizeof(b);
```

New Operators in C++

- C++ contains number of useful and powerful operators.
- The most operators and common to the operators in C.
- Now discussing the operators new to C++

Sr. No	Operator	Meaning
1	<<	Insertion Operator
2	>>	Extraction Operator
3	::	Scope Resolution Operator
4	new	Memory Allocation operator
5	delete	Memory release operator
6	endl	New line operator
7	setw	Field width operator